



**UNIVERSITY_{OF}
PORTSMOUTH**

School of Computing

Course:

**MSc Cybersecurity and Forensic Information
Technology**

Project Specification

Name:

Toyin Johnson Daniyan

Project Specification

1. Basic details

Student name:	Toyin Johnson Daniyan
Draft project title:	Automated Adversarial Testing of FAPI 2.0 Security Profile Conformance in Financial API Implementations
Course and year:	MSc Cybersecurity and Forensic Information Technology
Client contact name:	N/A
Project supervisor:	Paolo Serafino

2. Outline of the project environment

This project builds and tests an automated tool that checks whether a bank's or FinTech's API actually follows the FAPI 2.0 security rules. The Open Banking movement, driven by regulatory frameworks such as PSD2 (Payment Services Directive 2) in the UK and EU, has made APIs the primary mechanism through which banks, third-party payment providers, and financial aggregators share customer data. Prominent standards such as the UK Open Banking Standard and the Financial-grade API (FAPI) security profile have been established to govern these interactions; however, real-world implementations frequently deviate from these specifications, introducing exploitable vulnerabilities.

This project addresses that gap by designing, building, and evaluating an automated adversarial testing tool for FAPI 2.0 security profile conformance in financial API implementations, thereby contributing both a practical artefact and empirical evidence on the security posture of financial APIs.

3. The problem to be solved

Problem Statement

Manual conformance testing of financial API implementations against the FAPI 2.0 Security Profile is time-consuming, expensive, and inconsistent across practitioners. Existing general-purpose scanners such as OWASP ZAP and Burp Suite are not designed to assess FAPI 2.0-specific requirements, including PAR enforcement, PKCE mandation, sender-constrained token binding (mTLS/DPoP), and permitted client authentication methods, and instead focus on broader, standard-agnostic vulnerability classes. There is currently no widely available open-source tool that

systematically applies adversarial black-box test cases to determine whether a financial API implementation conforms to the FAPI 2.0 Security Profile.

Aim

To design and develop an automated adversarial black-box testing tool capable of assessing conformance to the FAPI 2.0 security profile in RESTful financial data APIs, and to evaluate its effectiveness against a set of deliberately vulnerable and real-world-style financial API endpoints.

Objectives

- Conduct a systematic literature review of API security vulnerabilities, existing scanning tools, and financial API security standards (OWASP API Top 10, FAPI, PSD2/Open Banking).
- Design a modular scanner architecture that supports authentication (OAuth 2.0 / OpenID Connect), endpoint enumeration, and automated payload injection.
- Implement scanner modules targeting the OWASP API Security Top 10 (2023) vulnerability classes, including BOLA, broken authentication, SSRF, and security misconfiguration.
- Develop financial-domain-specific checks aligned with the FAPI security profile and UK Open Banking Standard.
- Evaluate the scanner against a purpose-built vulnerable API lab environment (using tools such as crAPI) and optionally against publicly available sandbox APIs.
- Analyse results, measure detection accuracy (false positive/negative rates), and document findings in the dissertation.

Constraints

- Testing will be limited to environments explicitly authorised for security research; no production financial APIs will be scanned without written permission.
- The scanner will target RESTful APIs with JSON payloads; GraphQL and SOAP are out of scope for this iteration.
- The project must be completed within the MSc dissertation timeline.
- Computational resources will be limited to the student's personal hardware supplemented by free-tier cloud services.
- Ethical approval must be obtained before any data collection or live system testing.

4. Breakdown of tasks

Background Research

- Review the OWASP API Security Top 10 (2023), FAPI security profiles, PSD2 technical standards, and UK Open Banking specifications.
- Survey existing API scanning tools (ZAP, Burp Suite, Postman, 42Crunch) to identify capability gaps.
- Study OAuth 2.0, OpenID Connect, and JWT security best practices as they apply to financial APIs.
- Review academic literature on automated vulnerability detection, fuzzing techniques, and FinTech security incidents.

Tools and Technologies

- Languages: Python (primary scanner implementation), JavaScript (NextJs)/Node.js (vulnerable API lab).
- Libraries: Requests, httpx, PyJWT, Pydantic (scanner); FastAPI (lab API).
- Supporting tools: OWASP ZAP (baseline comparison), Postman / Newman (API testing reference), Docker (containerised lab environment).
- Version control and documentation: GitHub, LaTeX and Word for the dissertation.

Skills to Acquire

- In-depth knowledge of OAuth 2.0 attack
- API fuzzing and payload generation techniques
- Designing and conducting security experiments with measurable outcomes.

Design and Build

- A modular Python-based scanner with pluggable vulnerability check modules.
- A deliberately vulnerable financial API lab with documented known vulnerabilities for ground-truth evaluation.
- A reporting module that produces structured JSON and human-readable HTML vulnerability reports.

5. Project deliverables

Software Artefacts:

- Automated API Vulnerability Scanner (Python CLI).
- Vulnerable Financial API Lab (Docker-composed environment).
- Evaluation Dataset: structured results of scanner runs against the lab environment and any authorised sandbox APIs.
- HTML/JSON Vulnerability Report Generator.

Document:

- Dissertation

6. Requirements

Functional Requirements:

- Take a website address and an API spec file as input, and automatically find all the endpoints (URLs) to test
- Be able to log in using the security methods FAPI 2.0 requires (PKCE, PAR, and DPOp) before it starts testing
- Automatically try to "break" each endpoint's security in various ways, checking specifically for FAPI 2.0 rule violations (like whether PAR/PKCE are properly enforced, tokens are bound correctly, etc.)
- Generate a clear report showing: which endpoints failed, which specific rule they broke, how serious it is, and how to fix it
- The test lab (fake bank environment) must have at least one deliberately broken example for every rule being tested, so the tool has something real to catch

Non-Functional Requirements:

- Should be able to scan a 20-endpoint API in under 10 minutes on a normal computer
- Should be free/open-source, with instructions so anyone can set it up and reproduce the results
- Should track and report how often it wrongly flags something as broken when it isn't (false positives), and discuss that in the evaluation

7. Legal, Ethical, Professional, Social issues

Legal and Regulatory

All security testing will be conducted exclusively on systems owned or operated by I (lab environment) or on systems for which explicit written authorisation has been obtained. This ensures compliance with the Computer Misuse Act 1990 (UK). Any use of publicly available API sandboxes will be governed by the provider's terms of

service. No real customer financial data will be collected, processed, or stored; any synthetic data used in the lab will be entirely fabricated and will not constitute personal data under the UK GDPR.

Ethical Considerations

An ethics application will be submitted to the University of Portsmouth Ethics Committee prior to commencing any testing activity. The project does not involve human participants, sensitive personal data, or vulnerable populations, which is expected to make it a low-risk application.

Professional and Social Considerations

The outputs of this project may influence how financial institutions approach API security testing. Findings and tooling will be documented responsibly, and I will not publish specific vulnerability details for any real system without following coordinated disclosure processes. The scanner will include a usage disclaimer to discourage misuse.

8. Facilities and Resources

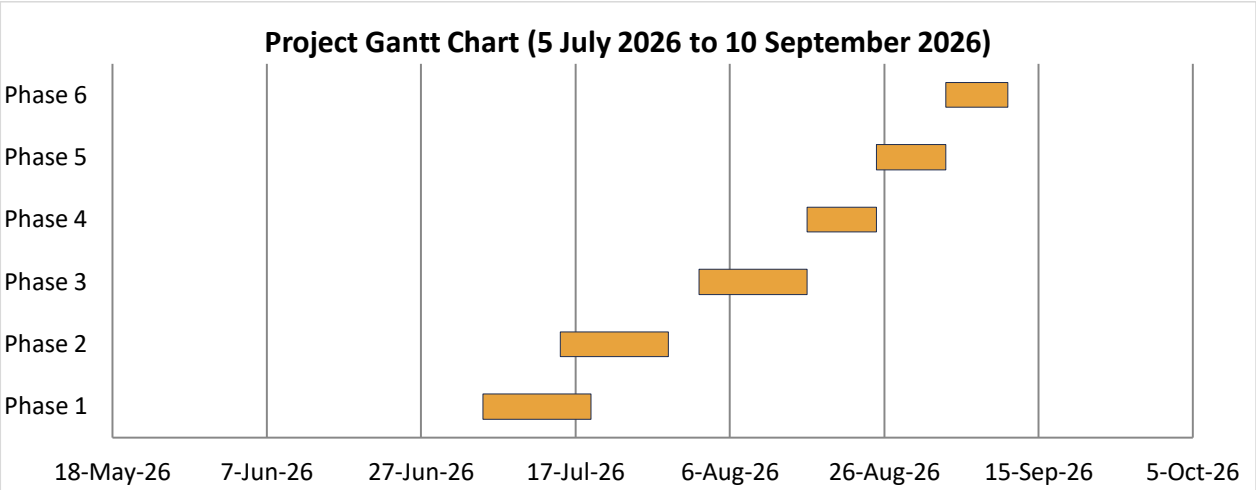
- Personal laptop (current hardware) running MacOS for scanner development and lab hosting.
- Docker Desktop for containerised lab environment deployment.
- GitHub for version control and CI/CD pipelines.
- University of Portsmouth library access for academic journals
- Free-tier cloud compute (GitHub Actions) for extended evaluation runs if local hardware proves insufficient.
- No significant costs are anticipated; all core tools and platforms are open-source or available under free academic licences.

9. Project plan

Phase	Description	Start Date	End Date	Duration (days)	Weeks
Phase 1	Literature review, ethics application, environment setup, and tool survey	05-Jul-26	18-Jul-26	14	2
Phase 2	Scanner architecture design and initial module implementation (authentication, endpoint enumeration)	15-Jul-26	01-Aug-26	14	2
Phase 3	Implementation of OWASP API Top 10 check modules and vulnerable lab API	02-Aug-26	15-Aug-26	14	2

Phase 4	Financial-domain-specific checks (FAPI / Open Banking compliance modules)	16-Aug-26	24-Aug-26	9	1.3
Phase 5	Evaluation phase: systematic testing, data collection, metric analysis	25-Aug-26	02-Sep-26	9	1.3
Phase 6	Dissertation writing, revisions, and final submission	03-Sep-26	10-Sep-26	8	1.1

Gantt Chart



Identified Risks and Mitigations

	Risk	Severity	Impact on Project	Mitigation
1.	Difficulty obtaining authorised external API targets for evaluation	Medium	Could narrow the evaluation to the mock lab only, limiting external validity of results	Primary evaluation will use the purpose-built mock Open Banking lab, supplemented by publicly available FAPI 2.0-conformant sandboxes (e.g. an OpenID Foundation certified authorization server sandbox, or an Open Banking ecosystem sandbox that has adopted FAPI 2.0) under their terms of service.
2.	Technical complexity of implementing and testing FAPI 2.0 mandated Oauth 2.0/OIDC flows exceeds initial estimates	High	Could delay or prevent completion of core scanner modules, threatening the central deliverable	Timeboxed implementation prioritizing mandatory FAPI 2.0 controls (PAR, PKCE) first; Sender-constrained token mechanisms (mTLS/DPoP) and less critical checks (e.g. JARM validation) treated as stretch targets if time is limited, rather than falling

				back to a non-FAPI authentication method
3.	Data loss due to hardware malfunction	High	Could destroy code, test lab configuration, or written chapters, potentially costing weeks of unrecoverable work	All code and configuration held in a private Git repository (GitHub) with regular, frequent commits; all written chapters and documents synced to the University of Portsmouth's cloud storage (OneDrive) with version history enabled; periodic manual backup to a separate physical drive for critical milestones.
4.	Illness during the project	Medium-High (depending on duration/timing)	Lost working days could delay milestones, with greater impact if illness coincides with critical-path task	Gantt chart timeline includes built-in contingency/buffer time ahead of key deadlines; task list prioritised (MoSCow style) so essential deliverables are front-loaded ahead of stretch goals. Early, transparent communication with supervisor if illness affects the timeline; awareness of the University's extenuating circumstances process for serious or prolonged illness
5.	Activities overrunning (tasks taking longer than planned)	Medium	Knock-on delays to later phases, compressing time available for evaluation, analysis, and writing	Gantt chart built with contingency buffers between phases rather than back to back scheduling; regular self review of progress against the timeline to catch slippage early; features/checks beyond the mandatory FAPI 2.0 controls treated as stretch goals that can be dropped without threatening the core deliverable.

10. Supervision meetings

Supervisor meeting will be held biweekly to be conducted via Microsoft Teams or in person at the University of Portsmouth campus, depending on availability. I will provide a brief written progress summary before each meeting to maximise the value of the session.

During periods of extended supervisor absence (e.g., annual leave or conference attendance), I will continue working to the agreed project plan, document any blockers, and raise urgent issues via email. A contingency contact within the School of Computing will be identified at the first supervision meeting.

11. Planned submission deadline

16th September, 2026

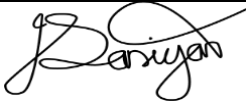
.....

12. Signatures

Signature:

Date:

Student

	4 th May, 2026

Client

Project supervisor